

# Capitolo 19 — PROC-003 — Deterministic Discovery & Durable Dispatch

|                    |                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------|
| <b>Pubblico</b>    | Lettori tecnici e non tecnici del WCM Process & Memory Book                                                |
| <b>Versione</b>    | FROZEN-19                                                                                                  |
| <b>Data master</b> | 2026-08-29                                                                                                 |
| <b>Stato</b>       | FROZEN                                                                                                     |
| <b>Source path</b> | wcm/documentation/process-memory-book/<br>chapters/19_proc_003_deterministic_discovery_durable_dispatch.md |
| <b>Source SHA</b>  | 3dc5feb                                                                                                    |
| <b>Release</b>     | 2026-09-04T09:44:50.631Z                                                                                   |

*GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.*

**Parte VI — Il Libro dei Processi WCM Stato:** FROZEN **Data:** 2026-08-29 **Scope:** WCM generale, domain-agnostic

## 19.0 Non tutto ciò che si ripete richiede intelligenza

Nel capitolo precedente abbiamo visto come WCM renda affidabile un workspace prima di usarlo come base operativa. Una volta verificata la fotografia del repository, emerge però un problema diverso:

**Come controlliamo periodicamente se esiste lavoro da eseguire senza spendere capacità cognitiva soltanto per scoprire che non c'è nulla da fare?**

Un sistema agentico ingenuo potrebbe fare questo:

```
OGNI N MINUTI
  ↓
SVEGLIA UN LLM
  ↓
CHIEDIGLI SE C'È LAVORO
  ↓
SE NON C'È, TERMINA
```

Funziona, ma usa ragionamento probabilistico per una domanda che spesso è meccanica. Se lo stato del lavoro è persistente e strutturato, la domanda:

```
ESISTE UN JOB READY?
```

non richiede interpretazione creativa. Richiede una verifica.

PROC-003 nasce da questa separazione:

```
CONTROL LOOP
≠
COGNITIVE LOOP
```

Il controllo periodico può essere deterministico. La capacità cognitiva viene attivata soltanto quando esiste lavoro reale.

---

## 19.1 Che cos'è PROC-003

**PROC-003 – Deterministic Discovery & Durable Dispatch** è il processo con cui WCM:

1. osserva periodicamente uno spazio autorizzato;
2. individua unità di lavoro realmente eleggibili;
3. evita di attivare un agente cognitivo quando non serve;
4. impedisce che lo stesso lavoro venga consegnato più volte;
5. quando serve esecuzione cognitiva, crea un **dispatch durevole** che trasporta il contesto necessario verso l'execution plane.

La sua domanda operativa non è:

**|** *“Come deve essere svolto il lavoro?”*

È:

**|** **“Esiste lavoro che deve essere consegnato, e devo creare un nuovo dispatch?”**

Questa distinzione separa due responsabilità:

```
DISCOVERY / CONTROL
=
SHOULD I DISPATCH?

EXECUTION / COGNITION
=
HOW DO I EXECUTE?
```

PROC-003 governa la prima. L'esecutore cognitivo governa la seconda entro il contratto ricevuto.

---

## 19.2 Perché il polling cognitivo è un anti-pattern

Immaginiamo un heartbeat che gira ogni ora. Per ventitré ore non c'è nuovo lavoro. Alla ventiquattresima compare un job **READY**.

Se ogni tick sveglia un modello AI soltanto per leggere lo stato, abbiamo prodotto ventitré attivazioni cognitive inutili e una utile.

Il problema non è soltanto economico. Ogni attivazione introduce anche:

- latenza;
- una nuova sessione;
- possibilità di interpretazione divergente;
- necessità di ricostruire contesto;
- telemetria aggiuntiva;

- superficie di failure;
- rischio di azioni duplicate.

Quando la domanda è deterministica, usare un LLM per rispondere significa trasformare un controllo booleano in un problema cognitivo.

La baseline di PROC-003 adotta quindi il principio:

```
SE IL CONTROLLO È MECCANICO,  
IL CONTROL LOOP DEVE RESTARE MECCANICO.
```

## 19.3 Il Sentinel deterministico

Il componente che esegue il controllo viene descritto nella baseline come **Sentinel**.

Il nome indica una funzione, non necessariamente un prodotto o un nuovo agente cognitivo:

```
SENTINEL  
=  
OSSERVARE  
+  
VERIFICARE ELEGGIBILITÀ  
+  
DECIDERE SE CREARE DISPATCH
```

Il Sentinel non deve decidere come risolvere il contenuto del job. Non deve riscrivere il goal, ampliare authority, allargare scope o inventare lavoro.

La restrizione è intenzionale: più una decisione è meccanica, più è utile mantenerla fuori dal nucleo probabilistico.

## 19.4 Il trigger periodico

PROC-003 è pensato per contesti in cui il sistema deve verificare a intervalli regolari se è comparso nuovo lavoro eleggibile.

Il trigger può quindi essere periodico:

```
TICK  
↓  
CONTROLLO DETERMINISTICO  
↓  
LAVORO READY?  
├─ NO → IDLE  
└─ YES → VALUTA DISPATCH
```

La periodicità non implica che ogni tick debba generare un'esecuzione completa. Il tick è soltanto un'occasione di osservazione.

L'assenza di lavoro è un esito valido. Non è una failure e non richiede un LLM per essere spiegata.

## 19.5 Prima della discovery: aggiornare la conoscenza del remoto

Se la discovery osserva un repository remoto attraverso una copia locale, deve prima aggiornare la propria conoscenza del remoto.

La baseline prevede:

```
git fetch origin
```

prima di decidere sulla base dello stato remoto osservato.

La ragione è la stessa incontrata in PROC-002:

```
REMOTE-TRACKING REF STALE  
→  
DISCOVERY STALE
```

Determinismo significa:

```
STESSO INPUT VERIFICATO  
→  
STESSA DECISIONE
```

Non significa che qualsiasi input, anche stale, produca una decisione affidabile.

Freshness e determinismo devono quindi lavorare insieme.

---

## 19.6 Che cosa viene osservato

Il Sentinel non deve scandire indiscriminatamente tutto ciò che esiste.

Gli input minimi di PROC-003 includono:

- repository e branch autorizzati;
- path di discovery;
- Service Job con stato persistente;
- assignee autorizzato;
- eventuale allowlist o routing autorizzato;
- project/workspace del runtime agentico;
- identificatore deterministico della versione remota osservata.

Anche la discovery ha quindi un perimetro:

```
DISCOVERY  
≠  
RICERCA OVUNQUE
```

Il confine riduce costo, rumore e rischio di trasformare un artefatto non operativo in lavoro eseguibile.

---

## 19.7 Il gate di eleggibilità

La presenza di un file o di un task non basta.

PROC-003 considera eleggibile il lavoro quando il Service Job soddisfa le condizioni previste dalla baseline, in particolare:

```
STATUS = READY
+
ASSIGNEE COERENTE
+
ROUTE / ALLOWLIST COERENTI
```

Come abbiamo visto con PROC-001:

```
ESISTE
≠
È ESEGUIBILE
```

Un job **HOLD**, **DONE** o fuori route non deve diventare un dispatch soltanto perché è visibile. Se nessun job è eleggibile, il processo termina:

```
IDLE_NO_READY
```

senza attivare capacità cognitive.

---

## 19.8 IDLE è un risultato corretto

Nei sistemi autonomi è facile associare valore alla quantità di attività prodotta. Un heartbeat che non crea nulla può sembrare inutile.

In PROC-003 è spesso il contrario.

```
OSSERVA
→
VERIFICA
→
NESSUN READY
→
NO-OP
```

La decisione di non fare nulla è parte del risultato.

Un sistema affidabile non deve produrre attività per dimostrare che funziona. Deve produrre la transizione corretta quando le condizioni la richiedono.

---

## 19.9 Quando compare un job READY

Quando il Sentinel trova un Service Job eleggibile, non deve ancora svegliare immediatamente il nucleo cognitivo.

Prima deve chiedersi:

```
QUESTO LAVORO
È GIÀ STATO DISPATCHATO
NELLA STESSA VERSIONE?
```

Qui entra in gioco la **dispatch key**.

La baseline raccomanda una chiave logica basata almeno su:

```
SERVICE_JOB_ID
+
REMOTE_SHA
```

Schema concettuale:

```
WCM_DISPATCH_KEY=<SERVICE_JOB_ID>:<REMOTE_SHA>
```

La coppia identifica non soltanto il lavoro, ma quel lavoro nella specifica versione osservata.

## 19.10 Perché il job ID da solo non basta

Supponiamo che un Service Job venga completato e poi riaperto esplicitamente dopo una modifica autorizzata.

Se la deduplicazione usasse soltanto l'ID, il sistema potrebbe concludere che quel job è già stato dispatchato e non deve esserlo più.

Ma:

```
JOB-42 @ SHA-A
```

è distinto da:

```
JOB-42 @ SHA-B
```

quando la versione remota è cambiata.

L'identità operativa deve quindi includere il lavoro e lo stato/versione osservata.

## 19.11 Il secondo gate: esiste già un dispatch equivalente?

Calcolata la dispatch key, il Sentinel verifica se esiste già un dispatch aperto equivalente.

```
JOB READY
↓
CALCOLA DISPATCH KEY
↓
DISPATCH APERTO EQUIVALENTE?
├ YES → NO-OP
└ NO  → CREA DURABLE DISPATCH
```

Questo gate protegge il sistema dal comportamento tipico dei loop periodici non idempotenti:

```
MINUTO 1 → DISPATCH A  
MINUTO 2 → DISPATCH B  
MINUTO 3 → DISPATCH C
```

Tre attivazioni cognitive per lo stesso lavoro.

PROC-003 impedisce questo effetto duplicato.

---

## 19.12 Idempotenza: un lavoro, un dispatch logico

Il protocollo direttamente collegato a PROC-003 è:

```
PROT-004 – Canonical Dispatch & Idempotency
```

La sua invariante centrale è:

```
UN LAVORO  
+  
UNA VERSIONE  
→  
UN DISPATCH LOGICO
```

L'idempotenza non vieta di ripetere il controllo. Impedisce che la ripetizione dello stesso input logico produca effetti duplicati indesiderati.

```
TICK 1: JOB READY @ SHA-X  
→ crea dispatch  
  
TICK 2: JOB READY @ SHA-X  
→ dispatch già aperto  
→ no-op  
  
TICK 3: JOB READY @ SHA-X  
→ dispatch già aperto  
→ no-op
```

Il controllo può ripetersi. L'effetto no.

---

## 19.13 Perché il tempo non basta per deduplicare

Un cooldown può ridurre la probabilità di duplicazione, ma non dimostra che il secondo dispatch sia equivalente al primo.

Il tempo risponde alla domanda:

```
QUANTO È PASSATO?
```

La deduplicazione deve rispondere:

```
È LO STESSO LAVORO  
NELLA STESSA VERSIONE  
GIÀ CONSEGNATO?
```

Sono problemi diversi.

Per questo PROT-004 richiede deduplicazione persistente e, quando disponibile, una chiave idempotente nativa del control plane. La baseline mantiene anche un controllo esplicito sui dispatch aperti per coprire retention o limiti del meccanismo nativo.

---

## 19.14 Che cos'è un durable dispatch

Quando non esiste un dispatch equivalente, PROC-003 crea un **durable canonical dispatch**.

La parola *durable* è essenziale.

Il dispatch non deve esistere soltanto come evento volatile del tipo “sveglia questo agente adesso”. Deve essere un oggetto persistente che possa essere:

- osservato;
- assegnato;
- rivendicato;
- correlato al Service Job;
- deduplicato;
- auditato;
- chiuso.

Possiamo pensarlo così:

```
DURABLE DISPATCH  
=  
ENVELOPE PERSISTENTE  
+ IDENTITÀ  
+ CONTESTO RISOLVIBILE  
+ CLAIM  
+ AUDIT
```

Il lavoro attraversa così il confine tra control plane ed execution plane senza affidare il proprio significato a un evento effimero.

---

## 19.15 Perché un generic wake può fallire

Un generic wake comunica, in sostanza:

```
ATTIVATI
```

Se il runtime non garantisce che il payload applicativo venga propagato nel contesto effettivo dell'esecutore, può accadere questo:



```
CONTROL PLANE
"HO INVIATO IL LAVORO"
```

```
EXECUTION PLANE
"SONO STATO SVEGLIATO,
MA NON SO PER COSA"
```

La baseline considera quindi superseded, per il dispatch di lavoro nel runtime validato, il generic wake come unico envelope quando la propagazione del payload non è garantita.

Il principio generale è indipendente dal prodotto:

**il segnale di attivazione non deve essere confuso con il contratto di lavoro.**

## 19.16 L'envelope minimo

PROT-004 richiede che il durable dispatch renda disponibili almeno gli elementi necessari a risolvere il lavoro corretto.

Concettualmente:

```
WAKE SOURCE
DISPATCH KEY
SERVICE JOB ID
SERVICE JOB PATH
PROJECT / SCOPE
BRANCH
REMOTE SHA
```

A questi si aggiungono, quando previsti, workspace, assignee e routing autorizzato.

Il dispatch non deve contenere tutta la conoscenza necessaria all'esecuzione. Deve contenere abbastanza informazione da permettere all'esecutore di raggiungere deterministicamente la source of truth e ricostruire il contesto corretto.

Questo è coerente con INDEX-FIRST:

```
ENVELOPE
→ IDENTITÀ + ROUTE
→ SOURCE OF TRUTH
→ RETRIEVAL PROGRESSIVO
```

Non:

```
ENVELOPE
→ COPIA DI TUTTA LA KB
```

## 19.17 Il dispatch non sostituisce il Service Job

La baseline separa chiaramente:

```
SERVICE JOB
=
CONTRATTO OPERATIVO
```

```
+ AUTHORITY
+ STATO DI VERITÀ

DURABLE DISPATCH
=
ENVELOPE
+ CLAIM
+ AUDIT
```

Il dispatch serve a consegnare il lavoro. Non acquisisce il diritto di ridefinirlo e non diventa automaticamente la nuova source of truth.

Questa separazione rende il trasporto sostituibile: un futuro runtime può materializzare il durable dispatch in modo diverso purché preservi identità, contesto risolvibile, deduplicazione, claim e audit.

## 19.18 Assignment wake: prima l'oggetto di lavoro, poi l'attivazione

Nella baseline implementata corrente, il durable dispatch viene materializzato come oggetto persistente assegnabile nel runtime agentico. L'assegnazione produce il normale wake nativo dell'esecutore.

```
DURABLE DISPATCH CREATO
↓
ASSIGNMENT
↓
WAKE NATIVO
↓
ESECUTORE RICEVE L'ENVELOPE
```

L'ordine è importante.

Non facciamo:

```
WAKE
→ poi speriamo di spiegare il lavoro
```

Facciamo:

```
LAVORO PERSISTITO COME DISPATCH
→ poi wake associato a quell'oggetto
```

L'attivazione nasce da un oggetto operativo già esistente e auditabile.

## 19.19 Il claim e il passaggio a PROC-001

Una volta svegliato, l'esecutore deve risolvere il Service Job corretto, verificare la base operativa quando richiesto e applicare il lifecycle del lavoro.

La relazione tipica è:

```
PROC-003
DISCOVERY + DISPATCH
↓
PROC-002
PRE-SYNC, SE APPLICABILE
↓
PROC-001
CLAIM + LIFECYCLE
```

PROC-003 non sostituisce PROC-001. Porta il lavoro al punto in cui PROC-001 può governarne presa in carico e chiusura.

---

## 19.20 READY non deve restare ambiguamente READY dopo il claim

Un failure mode della baseline riguarda il job dispatchato che rimane **READY** senza un claim coerente.

La deduplicazione può impedire nuovi dispatch, ma lo stato semantico del lavoro resta ambiguo.

Il percorso sano è:

```
READY
↓
DURABLE DISPATCH
↓
VALID CLAIM
↓
IN_PROGRESS
```

Il dispatch è un ponte, non uno stato terminale.

---

## 19.21 Il ciclo completo

Possiamo ora leggere PROC-003 dall'inizio alla fine:

```
TRIGGER PERIODICO
↓
SENTINEL DETERMINISTICO
↓
AGGIORNA CONOSCENZA REMOTA
↓
LEGGI IL PERIMETRO AUTORIZZATO
↓
SERVICE JOB READY E COERENTE?
├── NO
│   ↓
│   IDLE_NO_READY
│   0 ESECUZIONI COGNITIVE
└── YES
    ↓
```

```

CALCOLA DISPATCH KEY
↓
DISPATCH APERTO EQUIVALENTE?
├ YES → NO-OP
└ NO
  ↓
  CREA DURABLE DISPATCH
  ↓
  ASSIGNMENT WAKE NATIVO
  ↓
  ESECUTORE
  ↓
  PRE-SYNC / CLAIM
  ↓
  PROC-001

```

Il loop periodico può continuare a girare senza moltiplicare il lavoro cognitivo.

## 19.22 Che cosa è deterministico e che cosa resta cognitivo

PROC-003 è un esempio chiaro della separazione tra Deterministic Core e Cognitive Core.

Il lato deterministico può gestire:

- refresh dello stato remoto;
- lettura dello stato persistente;
- filtro **READY**;
- verifica di route e allowlist strutturate;
- calcolo della dispatch key;
- verifica di dispatch equivalente;
- creazione idempotente dell'envelope;
- no-op quando non serve agire.

Il lato cognitivo entra quando bisogna:

- comprendere il contenuto sostanziale del lavoro;
- scegliere come eseguirlo entro il contratto;
- interpretare materiale non riducibile a regole meccaniche;
- produrre output che richiedono ragionamento.

In sintesi:

DETERMINISTIC CORE:  
"DEVO ATTIVARE QUALCUNO?"

COGNITIVE CORE:  
"COME ESEGUO CORRETTAMENTE IL LAVORO?"

## 19.23 Failure mode principali

### Dispatch duplicato

Due dispatch equivalenti possono generare scritture concorrenti, doppia delivery, consumo duplicato di risorse e conflitti di stato. La risposta è deduplicazione persistente per identità logica, non semplice rallentamento del polling.

### Payload perso

Un wake tecnicamente riuscito non equivale a work delivery riuscita. Il durable dispatch rende verificabile che esista un oggetto di lavoro identificato e risolvibile.

### Route, branch o workspace incoerenti

Un dispatch tecnicamente perfetto può essere semanticamente sbagliato se consegna il lavoro all'esecutore o al perimetro errato. Il gate deve verificare route, assignee, progetto/workspace, branch e assenza di gate pendenti.

### Dispatch **in\_progress** senza continuazione viva

Un dispatch persistente deve mantenere uno stato significativo. La sua chiusura deve essere coerente con il lifecycle e l'acceptance del Service Job.

### Polling sproporzionato

Anche un controllo deterministico consuma risorse. La frequenza deve essere proporzionata al tempo di reazione necessario e al valore informativo prodotto; PROC-003 non impone una frequenza universale.

## 19.24 Tre oggetti, tre responsabilità

Il processo diventa più chiaro separando gli oggetti coinvolti:

```
SERVICE JOB
→ che cosa deve essere fatto
→ con quale authority
→ qual è lo stato operativo di verità

DURABLE DISPATCH
→ come il lavoro attraversa il confine di esecuzione
→ claim + envelope + audit

SENTINEL
→ se serve creare un dispatch adesso
```

A questi si aggiunge l'esecutore cognitivo:

```
COGNITIVE EXECUTOR
→ come svolgere il lavoro entro il Service Contract
```

Confondere questi ruoli genera gran parte dei problemi che PROC-003 vuole eliminare.

---

## 19.25 Processo e protocollo: PROC-003 non è PROT-004

La relazione è stretta, ma i due oggetti non sono equivalenti.

**PROC-003** descrive il flusso:

```
TRIGGER
→ DISCOVERY
→ ELIGIBILITY
→ DEDUP
→ DISPATCH
→ HANDOFF
```

**PROT-004** impone le invarianti che rendono affidabile il passaggio:

- dispatch durevole prima dell'attivazione cognitiva;
- un lavoro/versione, un dispatch logico;
- source of truth separata;
- envelope minimo;
- deduplicazione persistente;
- terminalità verificabile.

Quindi:

```
PROC-003
=
CHE COSA ACCADE E IN QUALE ORDINE

PROT-004
=
QUALI REGOLE DEVONO ESSERE RISPETTATE
```

---

## 19.26 La relazione con PROC-001 e PROC-002

I primi tre processi del Process Book formano una catena naturale quando un Service Job viene scoperto periodicamente e consegnato a un esecutore che opera su un workspace Git:

```
PROC-003
SCOPRE E CONSEGNA
↓
PROC-002
VERIFICA LA BASE OPERATIVA
↓
PROC-001
GOVERNA IL LIFECYCLE
```

Non è una sequenza universale obbligatoria per qualsiasi attività WCM. È una composizione contestuale.

Questo mostra una proprietà del Process Book: i processi sono leggibili separatamente, ma acquistano pieno valore quando vengono instradati e combinati secondo il caso concreto.

## 19.27 Esempio astratto completo

Immaginiamo tre Service Job:

```
SJ-A = DONE
SJ-B = HOLD
SJ-C = READY
```

Il Sentinel aggiorna la conoscenza del remoto e osserva il perimetro autorizzato.

**SJ-A** non è eleggibile perché terminale. **SJ-B** non è eleggibile perché non eseguibile. **SJ-C** è **READY** e route/assignee sono coerenti.

Il Sentinel calcola:

```
KEY = SJ-C:SHA-123
```

Non trova dispatch aperti equivalenti e crea il durable dispatch.

Al tick successivo **SJ-C** è ancora visibile nella stessa versione, ma il dispatch **SJ-C:SHA-123** esiste già.

Il Sentinel esegue:

```
NO-OP
```

Non nasce una seconda esecuzione cognitiva.

Quando l'esecutore effettua il claim, PROC-001 governa il passaggio verso **IN\_PROGRESS** e, dopo acceptance verificata, verso **DONE**.

Il significato del processo può essere riassunto così:

```
OSSERVARE SPESSO
NON SIGNIFICA
ESEGUIRE SPESSO.
```

## 19.28 Stesso ID, nuova versione

Supponiamo che un job venga riaperto esplicitamente dopo una modifica autorizzata.

Prima:

```
SJ-C @ SHA-123
→ dispatch completato
```

Poi:

```
SJ-C @ SHA-987
→ READY
```

La nuova dispatch key è diversa:

```
SJ-C:SHA-987
```

Il sistema distingue così una nuova versione legittima del lavoro da un tick duplicato sul lavoro precedente.

## 19.29 Continuità organizzativa senza AI sempre accesa

In un'organizzazione agentica, il tempo passa anche quando nessun agente cognitivo è attivo. Nuovi job possono comparire, authority può rendere un lavoro **READY**, una versione remota può cambiare.

Il sistema ha quindi bisogno di osservazione continua o periodica.

Ma non deve confondere:

```
ESSERE VIGILE
```

con:

```
TENERE UN LLM CONTINUAMENTE ATTIVO
```

PROC-003 costruisce questo ponte. WCM può restare operativo come organizzazione persistente mentre il Cognitive Core viene attivato soltanto quando esiste una ragione concreta.

## 19.30 Aumentare il determinismo senza eliminare l'AI

PROC-003 non cerca di rendere deterministica l'intera organizzazione. Fa qualcosa di più preciso:

**| sposta fuori dall'AI le decisioni che non hanno bisogno dell'AI.**

Il risultato è un'architettura ibrida:

```
DETERMINISTIC CORE
→ discovery
→ eligibility strutturata
→ dedup
→ dispatch

COGNITIVE CORE
→ comprensione
→ scelta operativa nel perimetro
→ produzione dell'output
```



Il valore non sta nel sostituire l'uno con l'altro, ma nel collocarli nel punto giusto.

## 19.31 Maturity: che cosa significa VALIDATED

Nel Process Register corrente, PROC-003 è classificato:

VALIDATED

La qualifica deriva da evidence concreta su una implementazione controllata che ha verificato, tra gli altri elementi:

- polling deterministico senza esecuzioni LLM in idle;
- un solo durable dispatch per la stessa identità logica;
- wake nativo associato al dispatch;
- contesto ricevuto dall'esecutore;
- lifecycle del Service Job completato;
- tick successivi chiusi senza nuovi dispatch equivalenti.

Questa evidence sostiene la baseline metodologica corrente.

Non significa che ogni runtime, ogni scala, ogni frequenza di polling o ogni infrastruttura futura sia già stata validata universalmente.

BASELINE VALIDATA NEL PERIMETRO OSSERVATO  
≠  
GARANZIA UNIVERSALE

## 19.32 Che cosa non dobbiamo dedurre

PROC-003 non implica che:

- ogni progetto debba usare polling;
- ogni runtime debba usare lo stesso prodotto agentico;
- ogni dispatch debba essere materializzato nello stesso modo;
- ogni job debba essere controllato con la stessa frequenza;
- ogni attività debba attraversare un Service Job;
- il Cognitive Core possa essere eliminato;
- un dispatch persistente acquisisca authority autonoma;
- l'idempotenza elimini ogni failure mode dei sistemi distribuiti.

Il processo definisce una soluzione generale a un problema preciso:

DISCOVERY PERIODICA  
+  
ATTIVAZIONE COGNITIVA ON-DEMAND  
+  
DISPATCH DUREVOLE  
+  
DEDUPLICAZIONE PERSISTENTE

---

## 19.33 La domanda progettuale che introduce

Di fronte a un'automazione ricorrente, la domanda istintiva può essere:

| *“Quale agente devo svegliare?”*

PROC-003 suggerisce di chiedere prima:

| **“La decisione di svegliare qualcuno richiede davvero intelligenza?”**

Se la risposta è no, una componente deterministica può osservare lo stato e attivare AI soltanto quando le condizioni sono soddisfatte.

Passiamo da:

```
AI SEMPRE ACCESA  
CHE CONTROLLA SE SERVE AI
```

A:

```
CONTROLLO DETERMINISTICO  
CHE ATTIVA AI  
QUANDO SERVE DAVVERO
```

---

## 19.34 In sintesi

PROC-003 governa il confine tra **osservazione periodica** ed **esecuzione cognitiva**.

La sequenza essenziale è:

```
TICK  
→ REFRESH REMOTO  
→ DISCOVERY NEL PERIMETRO  
→ JOB READY?  
→ DISPATCH KEY  
→ DEDUP  
→ DURABLE DISPATCH  
→ ASSIGNMENT  
→ CLAIM  
→ EXECUTION
```

Le invarianti fondamentali sono:

1. nessun LLM deve essere attivato soltanto per una verifica meccanica quando esiste una primitiva deterministica sufficiente;
2. un job è eleggibile soltanto quando stato persistente e routing lo rendono realmente eseguibile;
3. lo stesso job nella stessa versione deve produrre un solo dispatch logico;
4. il dispatch deve essere durevole e trasportare un'identità di lavoro risolvibile;
5. il Service Job resta source of truth di contratto, authority e stato;
6. il dispatch non sostituisce il lifecycle governato da PROC-001;
7. il controllo può ripetersi senza moltiplicare gli effetti.

Il risultato è un sistema capace di restare vigile senza tenere costantemente acceso il proprio livello cognitivo.

Ed emerge una proprietà architetturale che ritroveremo più avanti:

**WCM non cerca di rendere deterministica l'intelligenza. Cerca di rendere deterministico tutto ciò che non ha bisogno di essere intelligente.**

---

## Source map del capitolo

Fonti canoniche principali utilizzate per la verifica tecnica:

- `WCM_AGENT_START.md`;
- `wcm/process-book/PROCESS_REGISTER.md`;
- `wcm/process-book/processes/PROC-003_DETERMINISTIC_DISCOVERY_DURABLE_DISPATCH.md`;
- `wcm/process-book/protocols/PROT-004_CANONICAL_DISPATCH_IDEMPOTENCY.md`;
- continuità editoriale con i Capitoli 17 e 18 FROZEN.

## Maturity qualifier

`PROC-003` e `PROT-004` risultano **VALIDATED** nella baseline WCM corrente. La validazione sostiene il modello operativo nel perimetro osservato e non costituisce un claim di universalità per ogni runtime, scala o infrastruttura futura.